Arrays Data Structure

From Scratch to Advanced — Concepts, Patterns, Classic Interview Problems, and Tips

---

A complete reference covering array fundamentals, core problem-solving patterns, 12 classic interview problems with solutions, and a general framework for building coding logic.

1. What is an Array?

A contiguous block of memory storing elements of the same type, accessed by index in O(1) time because the address is computed directly:

```
address = base_address + index * element_size
```

Index	0	1	2	3	4
Value	10	20	30	40	50

Static vs Dynamic Arrays

- **Static arrays** (C, Java arrays): fixed size, set at creation time.
- **Dynamic arrays** (Python `list`, Java `ArrayList`, C++ `vector`, JS `Array`): resize automatically by allocating a new, larger block (usually 2x) and copying elements over when full.

2. Time Complexity Cheat Sheet

Operation	Static Array	Dynamic Array
Access by index	O(1)	O(1)
Search (unsorted)	O(n)	O(n)
Search (sorted, binary search)	O(log n)	O(log n)
Insert/delete at end	N/A (fixed)	O(1) amortized
Insert/delete at start/middle	O(n)	O(n)

Why "amortized" O(1) for dynamic array append? Most appends are O(1), but occasionally the array is full and must resize (O(n) copy). Averaged over many operations, it still comes out to O(1) per append.

3. Basic Operations (Python)

```
arr = [10, 20, 30, 40, 50]

arr[2]                  # access -> 30
arr.append(60)          # insert at end -> O(1) amortized
arr.insert(1, 15)       # insert at index -> O(n), shifts elements right
arr.pop()               # remove last -> O(1)
arr.pop(1)              # remove at index -> O(n), shifts elements left
30 in arr               # search -> O(n)
arr[1:3]                # slicing -> O(k)
```

Traversal

```
for x in arr:
    print(x)
```

2D Arrays / Matrices

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]
matrix[1][2]   # -> 6, row 1 col 2
```

4. Core Techniques (The Toolbox)

These four patterns solve roughly 80% of array interview questions.

A. Two Pointers

Use when the array is sorted, or you're comparing elements from both ends, or partitioning in place.

```
def two_sum_sorted(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return [left, right]
        elif s < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]
```

B. Sliding Window

Use for subarray/substring problems with a contiguous window — max/min sum, longest/shortest subarray meeting a condition.

```
def max_sum_subarray_k(arr, k):
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i-k]   # slide: add new, drop old
        max_sum = max(max_sum, window_sum)
    return max_sum
```

C. Prefix Sum

Use for range-sum queries or subarray-sum problems — precompute cumulative sums so any range query is O(1).

```
def build_prefix(arr):
    prefix = [0] * (len(arr) + 1)
    for i, x in enumerate(arr):
        prefix[i+1] = prefix[i] + x
    return prefix

# sum of arr[l..r] inclusive:
# prefix[r+1] - prefix[l]
```

D. Hashing (dict/set) for O(1) lookup

Use to trade space for time — turn an $O(n^2)$ brute force into O(n).

```
def two_sum(arr, target):
    seen = {}  # value -> index
```

```
for i, x in enumerate(arr):
    complement = target - x
    if complement in seen:
        return [seen[complement], i]
    seen[x] = i
return [-1, -1]
```

5. Classic Interview Problems

1. Two Sum

Given array + target, return indices of two numbers that add to target.

```
def two_sum(arr, target):
    seen = {}
    for i, x in enumerate(arr):
        complement = target - x
        if complement in seen:
            return [seen[complement], i]
        seen[x] = i
    return [-1, -1]
```

Key insight: Hashmap trades O(n) space for O(n) time instead of O(n^2) brute force.

2. Maximum Subarray (Kadane's Algorithm)

Find the contiguous subarray with the largest sum.

```
def max_subarray(arr):
    max_ending_here = max_so_far = arr[0]
    for x in arr[1:]:
        max_ending_here = max(x, max_ending_here + x)
        max_so_far = max(max_so_far, max_ending_here)
    return max_so_far
```

Key insight: If the running sum goes negative, drop it and restart from the current element. O(n) time, O(1) space.

3. Best Time to Buy and Sell Stock

Find max profit from one buy + one sell.

```
def max_profit(prices):
    min_price = float('inf')
    max_profit = 0
    for p in prices:
        min_price = min(min_price, p)
        max_profit = max(max_profit, p - min_price)
    return max_profit
```

Key insight: Track the minimum price seen so far; profit is always current price minus that minimum.

4. Product of Array Except Self

Return an array where each element is the product of all others, without division, in O(n).

```
def product_except_self(nums):
    n = len(nums)
    res = [1] * n
    left = 1
```

```
    for i in range(n):
        res[i] = left
        left *= nums[i]
    right = 1
    for i in range(n-1, -1, -1):
        res[i] *= right
        right *= nums[i]
    return res
```

Key insight: Compute prefix products left-to-right, then multiply in suffix products right-to-left.

5. Move Zeroes

Move all zeros to the end, keep relative order of non-zeros, in place.

```
def move_zeroes(nums):
    insert_pos = 0
    for x in nums:
        if x != 0:
            nums[insert_pos] = x
            insert_pos += 1
    for i in range(insert_pos, len(nums)):
        nums[i] = 0
```

Key insight: Two-pass in-place write pointer — overwrite non-zeros forward, then fill the rest with zeros.

6. Merge Intervals

Merge all overlapping intervals.

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    result = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
        else:
            result.append([start, end])
    return result
```

Key insight: Sorting by start time turns an O(n^2) overlap check into a single linear sweep.

7. Container With Most Water

Two vertical lines form a container; find the pair with max area.

```
def max_area(height):
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        best = max(best, min(height[left], height[right]) * (right - left))
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best
```

Key insight: Always move the pointer at the shorter line — moving the taller one can never increase the area.

8. Rotate Array

Rotate an array right by k steps, in place.

```
def rotate(nums, k):
    n = len(nums)
    k %= n
    nums.reverse()
    nums[:k] = reversed(nums[:k])
    nums[k:] = reversed(nums[k:])
```

Key insight: Reverse the whole array, then reverse each of the two resulting segments.

9. Find Duplicate Number

Array of n+1 ints in range [1,n]; find the duplicate without modifying the array or using extra space.

```
def find_duplicate(nums):
    slow = fast = nums[0]
    while True:
```

```
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break
    slow2 = nums[0]
    while slow != slow2:
        slow = nums[slow]
        slow2 = nums[slow2]
    return slow
```

Key insight: Floyd's cycle detection — treat the array as an implicit linked list via indices. O(n) time, O(1) space.

10. Trapping Rain Water

Compute how much water is trapped between bars after raining.

```
def trap(height):
    left, right = 0, len(height) - 1
    left_max = right_max = water = 0
    while left < right:
        if height[left] < height[right]:
            left_max = max(left_max, height[left])
            water += left_max - height[left]
            left += 1
        else:
            right_max = max(right_max, height[right])
            water += right_max - height[right]
            right -= 1
    return water
```

Key insight: Two pointers track the tallest bar seen so far from each side; the shorter side bounds trapped water.

11. Subarray Sum Equals K

Count the number of contiguous subarrays that sum to k.

```
def subarray_sum(nums, k):
    count = 0
    curr_sum = 0
    prefix_counts = {0: 1}
    for x in nums:
        curr_sum += x
        count += prefix_counts.get(curr_sum - k, 0)
        prefix_counts[curr_sum] = prefix_counts.get(curr_sum, 0) + 1
    return count
```

Key insight: Combines prefix sum with a hashmap of prefix-sum frequencies.

12. Sort Colors (Dutch National Flag)

Sort an array of 0s, 1s, 2s in place, in one pass.

```
def sort_colors(nums):
    low, mid, high = 0, 0, len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1; mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
```

Key insight: Three pointers partition the array into 0s, 1s, and 2s regions in a single pass.

6. Tips & Tricks for Solving Array Problems

- **Always check: is it sorted?** Sorted → binary search or two pointers unlocks O(log n) or O(n) solutions.
- **Brute force first, then optimize.** State the $O(n^2)$ solution out loud, then ask "what am I recomputing?" — that recomputation is usually solved by a hashmap or prefix sum.
- **Two pointers from opposite ends** when comparing/combining the smallest and largest remaining elements (sorted arrays, palindrome checks, container problems).
- **Sliding window** whenever the problem says "contiguous subarray/substring" and asks for max/min/count satisfying some condition — grow the window, shrink it when a constraint is violated.
- **Prefix sums** turn any "sum of range" query into O(1) after O(n) preprocessing.
- **HashMap/Set trades space for time.** Nested loop searching for a value? Ask "can I remember what I've seen?"
- **In-place tricks:** use the array itself as a marker (negate values, swap to correct position) for O(1) space — common in "find missing/duplicate number" problems.
- **Sorting first** often simplifies a problem (merge intervals, 3Sum) at the cost of O(n log n).
- **Watch for off-by-one errors** at loop boundaries — trace a small 3-4 element example by hand.
- **Edge cases to always check:** empty array, single element, all same elements, sorted, reverse sorted, negative numbers, duplicates.
- **For "in-place" requirements**, think about iteration direction (forward vs backward) to avoid overwriting data you still need.
- **Cycle detection (Floyd's)** is a hidden gem for "array as implicit linked list" problems.
- **Kadane's pattern** (running value that resets when unhelpful) generalizes beyond sums — max product subarray, longest subarray patterns, etc.
- **Explain time/space complexity out loud** as you code — interviewers weight this as much as correctness.

7. Building Coding Logic — A General Framework

This process works for any DSA problem, not just arrays.

Step 1 — Understand before solving

Restate the problem in your own words. Identify: inputs, outputs, constraints (size, value range), and edge cases. Manually work a small example by hand — 3-5 elements — before writing any code.

Step 2 — Brute force first, always

Ask "what's the dumbest solution that obviously works?" State its time complexity out loud. This gives you a correctness baseline and reveals where the waste is.

Step 3 — Find what's being recomputed

- Recomputing a sum repeatedly → **prefix sum**
- Recomputing "have I seen this value" → **hashmap/set**
- Recomputing a min/max over a shrinking window → **sliding window / monotonic deque**
- Re-scanning from the start each time → **two pointers**

Step 4 — Match the problem shape to a known pattern

Signal in problem	Likely pattern
"sorted array"	binary search, two pointers
"contiguous subarray/substring"	sliding window
"count subarrays/pairs with sum/property"	prefix sum + hashmap
"find pair/triplet"	two pointers (after sorting) or hashmap
"next greater/smaller element"	monotonic stack
"kth largest/smallest", "top k"	heap
"overlapping ranges"	sort by start, sweep
"graph of choices, all paths"	backtracking/DFS
"optimal value, choices build on subproblems"	dynamic programming
"linked list/array cycle"	fast & slow pointers

Step 5 & 6 — Code, trace, and analyze

Code the optimized version, then trace it by hand on your small example before running it. Analyze complexity explicitly — say it out loud, even alone. This trains the habit interviewers want to see.

How to Actually Build This Skill

- **Volume with spaced repetition beats one-shot difficulty.** Solve ~150-200 problems across patterns (NeetCode 150 / Blind 75 are well-curated) rather than 500 random ones.
- **Study by pattern, not by random difficulty.** Do 8-10 sliding window problems in a row, then 8-10 two-pointer problems, etc.
- **After solving, ask "could I have recognized this pattern faster?"** Write one sentence: "sorted + pair search → two pointers."
- **Re-solve problems after a gap** (1-2 weeks later) without looking at your old solution.
- **Explain your approach out loud before coding**, even alone.
- **When stuck for more than ~10-15 min**, look at just the name of the technique (not the solution) and try again.

8. 3-Week Practice Plan

Week 1 — Two Pointers, Sliding Window, Prefix Sum

Day	Pattern	Problems
1	Two Pointers (basics)	Two Sum II (sorted), Valid Palindrome, Reverse Array
2	Two Pointers (advanced)	Container With Most Water, 3Sum, Sort Colors
3	Sliding Window (fixed size)	Max Sum Subarray of Size K, Average of Subarrays
4	Sliding Window (variable size)	Longest Substring Without Repeating Chars, Minimum Size Subarray Sum
5	Prefix Sum	Range Sum Query, Subarray Sum Equals K
6	Mixed review	Redo 2 hardest from days 1-5 without looking
7	Rest / catch up	Re-attempt anything you got wrong

Week 2 — Hashing, Sorting Tricks, Stacks

Day	Pattern	Problems
8	Hashmap/Set	Two Sum, Group Anagrams, Contains Duplicate
9	Hashmap (frequency)	Top K Frequent Elements, Longest Consecutive Sequence
10	Sort-first tricks	Merge Intervals, Non-overlapping Intervals
11	Cyclic sort / in-place index tricks	Find Missing Number, Find Duplicate Number, First Missing Positive
12	Monotonic Stack	Next Greater Element, Daily Temperatures
13	Monotonic Stack (harder)	Trapping Rain Water, Largest Rectangle in Histogram
14	Rest / mixed review	Redo 2 hardest from week 2

Week 3 — Kadane's Variants, Binary Search, Combine Patterns

Day	Pattern	Problems
15	Kadane's family	Maximum Subarray, Maximum Product Subarray
16	Binary Search on answer	Search in Rotated Sorted Array, Find Min in Rotated Array
17	Matrix/2D arrays	Rotate Image, Spiral Matrix, Set Matrix Zeroes
18	Greedy + arrays	Jump Game, Gas Station
19	Combine 2+ patterns	Product of Array Except Self, 3Sum Closest
20	Mock interview	3 random problems from days 1-19, cold, 25 min each, explain out loud

21	Review & reflect	Note which pattern took longest to recognize; redo those 3

Rules to Actually Make This Work

- **Timebox each problem to 20-25 min** before peeking at a hint.
- **After solving, write one line:** "signal → pattern" in a running notes file.
- **Days 6, 14, 21 are non-negotiable** — spaced repetition converts memorization into recognition.
- **Say your approach out loud before coding**, every time, even alone.